

ADVANCED PROGRAMMING

ASSIGNMENT 7

NAME- BHITALI BORUAH

ROLLNO-CSB24045

Assignment—7

```
from typing import List, Dict, Set, Union
from collections import defaultdict
from functools import reduce

def total_time_per_user(logs: List[Dict[str, Union[str, float]]]) -> Dict[str, float]:
    time_per_user = defaultdict(float)

    def accumulate_times(acc, log):
        acc[log["user"]] += log["duration"]
        return acc

    result = reduce(accumulate_times, logs, time_per_user)
    return dict(result)

def total_time_per_user_comprehension(logs: List[Dict[str, Union[str, float]]]) -> Dict[str, float]:
    users = {log["user"] for log in logs}
    return {
        user: sum(log["duration"] for log in logs if log["user"] == user)
        for user in users
    }

def most_active_users(logs: List[Dict[str, Union[str, float]]], k: int) -> List[str]:
    user_times = total_time_per_user(logs)
    sorted_users = sorted(
        user_times.items(),
        key=lambda x: x[1],
        reverse=True
   )[:k]
    return [user for user, _ in sorted_users]

def unique_actions(logs: List[Dict[str, Union[str, float]]]) -> Set[str]:
    return {log["action"] for log in logs}

def total_activity_time(logs: List[Dict[str, Union[str, float]]]) -> float:
    return reduce(
        lambda total, log: total + log["duration"],
        logs,
        0.0
    )
```

```

def group_by_user(logs: List[Dict[str, Union[str, float]]]) -> Dict[str, List[Dict]]:
    user_activities = defaultdict(list)
    for log in logs:
        user_activities[log["user"]].append(log)
    return dict(user_activities)

def get_action_summary(logs: List[Dict[str, Union[str, float]]]) -> Dict[str, Dict]:
    action_groups = defaultdict(list)
    for log in logs:
        action_groups[log["action"]].append(log["duration"])

    return {
        action: {
            "total_time": sum(times),
            "count": len(times),
            "average_time": sum(times) / len(times) if times else 0,
            "max_time": max(times) if times else 0,
            "min_time": min(times) if times else 0
        }
        for action, times in action_groups.items()
    }

def filter_activities(logs: List[Dict[str, Union[str, float]]],
                     min_duration: float = 0) -> List[Dict]:
    return [log for log in logs if log["duration"] >= min_duration]

if __name__ == "__main__":
    sample_logs = [
        {"user": "CS001", "action": "YouTube", "duration": 45.5},
        {"user": "CS002", "action": "VS Code", "duration": 120.0},
        {"user": "CS001", "action": "Stack Overflow", "duration": 30.2},
        {"user": "CS003", "action": "YouTube", "duration": 60.0},
        {"user": "CS002", "action": "GitHub", "duration": 45.0},
        {"user": "CS001", "action": "YouTube", "duration": 25.5},
        {"user": "CS004", "action": "Zoom", "duration": 90.0},
        {"user": "CS003", "action": "VS Code", "duration": 75.0},
        {"user": "CS002", "action": "YouTube", "duration": 35.0},
    ]

    print("Activity Log Analyzer")
    print("=" * 50)

    user_times = total_time_per_user(sample_logs)

```

```

print("\n1. Total Time Per User:")
for user, time in sorted(user_times.items(), key=lambda x: x[1],
reverse=True):
    print(f"    {user}: {time:.1f} minutes")

print("\n2. Top 3 Most Active Users:")
top_users = most_active_users(sample_logs, 3)
for i, user in enumerate(top_users, 1):
    print(f"    {i}. {user} ({user_times[user]:.1f} minutes)")

print("\n3. Unique Actions:")
actions = unique_actions(sample_logs)
for action in sorted(actions):
    print(f"    • {action}")

print(f"\n4. Total Activity Time: {total_activity_time(sample_logs):.1f}
minutes")

print("\n5. Action Summary:")
summary = get_action_summary(sample_logs)
for action, stats in sorted(summary.items()):
    print(f"    {action}:")
    print(f"        Total: {stats['total_time']:.1f} min")
    print(f"        Count: {stats['count']}")
    print(f"        Avg: {stats['average_time']:.1f} min")

print("\n6. Activities longer than 60 minutes:")
long_activities = filter_activities(sample_logs, 60)
for activity in long_activities:
    print(f"    {activity['user']} - {activity['action']}:
{activity['duration']} min")

print("\n7. Activities grouped by user:")
user_groups = group_by_user(sample_logs)
for user, activities in sorted(user_groups.items()):
    print(f"    {user}: {len(activities)} activities")

```

Output—

Activity Log Analyzer

=====

1. Total Time Per User:

CS002: 200.0 minutes
CS003: 135.0 minutes
CS001: 101.2 minutes
CS004: 90.0 minutes

2. Top 3 Most Active Users:

1. CS002 (200.0 minutes)
2. CS003 (135.0 minutes)
3. CS001 (101.2 minutes)

3. Unique Actions:

- GitHub
- Stack Overflow
- VS Code
- YouTube
- Zoom

4. Total Activity Time: 526.2 minutes

5. Action Summary:

GitHub:

Total: 45.0 min

Count: 1

Avg: 45.0 min

Stack Overflow:

Total: 30.2 min

Count: 1

Avg: 30.2 min

VS Code:

Total: 195.0 min

Count: 2

Avg: 97.5 min

YouTube:

Total: 166.0 min

Count: 4

Avg: 41.5 min

Zoom:

Total: 90.0 min

Count: 1

Avg: 90.0 min

6. Activities longer than 60 minutes:

CS002 - VS Code: 120.0 min

CS003 - YouTube: 60.0 min

CS004 - Zoom: 90.0 min

CS003 - VS Code: 75.0 min

7. Activities grouped by user:

CS001: 3 activities

CS002: 3 activities

CS003: 2 activities

CS004: 1 activities